Cascading Temporal Memory Architecture:

A Biologically-Inspired Framework for Persistent, Adaptive Memory in AI Systems

gutbut.com Research
research@gutbut.com
https://gutbut.com

March 2026

Abstract

Contemporary large language models (LLMs) suffer from a fundamental architectural constraint: the absence of persistent, user-adaptive memory. Despite advances in reasoning and generation, every conversation begins from a blank slate. Context windows offer a fixed-length buffer but impose hard boundaries on temporal continuity, provide no mechanism for importance-weighted retention, and fail to model evolving information salience over time. We introduce the **Cascading Temporal Memory Architecture (CTMA)**, a novel three-tier memory framework that draws on principles from cognitive psychology, neuroscience, and information theory to construct a longitudinal, user-specific knowledge graph with biologically-inspired decay dynamics. CTMA implements a memory lifecycle spanning ephemeral, cascading, and permanent storage tiers, governed by a temporal decay function modulated by behavioral signals, recency, frequency, and contextual importance. We present the mathematical formalization, system architecture, data model, and preliminary performance characteristics of CTMA as implemented within the `gutbut.com` platform. Empirical observations from prior iterations of the architecture (2023–2024), conducted across approximately 2,500 active users, indicated a 34% improvement in user satisfaction over memoryless baselines and 19% over naïve retrieval-augmented generation, with decay-calibrated salience correlating strongly with user-perceived relevance ($\rho = 0.71$, $p < 0.001$). The platform has since been rebuilt from the ground up, and a formal validation study on the current architecture is underway. We argue that the longitudinal dataset generated by this architecture constitutes a defensible technical moat for AI applications requiring genuine user understanding over extended time horizons.

Keywords: memory architecture, temporal decay, longitudinal datasets, persistent AI memory, user modeling, knowledge graphs, cascading retention, cognitive-inspired AI

1 Introduction

1.1 The Amnesia Problem in Modern AI

The current generation of large language models represents a remarkable achievement in artificial intelligence. Models such as GPT-4 [OpenAI, 2023], Claude [Anthropic, 2024], and Gemini [Team et al., 2023] demonstrate near-human performance across a wide range of cognitive tasks. Yet they share a crippling limitation: they cannot remember. Each session is stateless. Each conversation is disconnected from every prior interaction. The user must re-establish context, re-explain preferences, and re-teach the system what it should already know.

This is not a minor inconvenience—it is a fundamental architectural failure that prevents AI systems from building the deep, longitudinal understanding that underpins genuine personalization, anticipatory assistance, and trusted long-term relationships between humans and

intelligent systems. A physician who forgets every patient encounter, an advisor who cannot recall prior counsel, a collaborator who loses all shared context between sessions—these would be functionally useless in practice. Yet this is precisely how we deploy conversational AI today.

1.2 Why Context Windows Are Not Memory

The dominant approach to providing LLMs with session-level context is the attention-based context window, first formalized in the Transformer architecture [Vaswani et al., 2017]. The self-attention mechanism computes relevance scores across all token positions within the window, enabling rich in-context reasoning. However, the context window is a *buffer*, not a *memory system*. The distinction is critical:

(i) **Fixed capacity.** Context windows have hard token limits (4K–200K tokens). Information outside this window is irretrievably lost.

(ii) **No decay dynamics.** All tokens within the window receive equal treatment. There is no mechanism to prioritize recent, frequent, or important information over stale or trivial tokens.

(iii) **No cross-session persistence.** When the session ends, the window is cleared. There is no carryover between conversations.

(iv) **No behavioral modulation.** The window does not adapt based on user behavior—what they click, revisit, correct, or ignore has no influence on retention priority.

These limitations are not engineering oversights; they are consequences of the Transformer's fundamental design as a stateless function mapping input sequences to output sequences, with no internal state persisting across invocations.

1.3 Contributions

This paper makes the following contributions:

(1) We formalize the **Cascading Temporal Memory Architecture (CTMA)**, a three-tier memory system with biologically-inspired decay dynamics (Section 4).

(2) We present a **mathematical framework** for memory salience that unifies temporal decay, frequency reinforcement, behavioral signals, and contextual co-activation (Section 4).

(3) We describe a **complete system architecture** including extraction, storage, decay, and retrieval pipelines (Sections 5–7).

(4) We report **empirical observations** from prior iterations of the architecture deployed to approximately 2,500 users (Section 10).

(5) We articulate how the **longitudinal dataset** generated by CTMA creates a defensible technical moat (Section 8).

2 Related Work

2.1 External Memory for Neural Networks

The problem of augmenting neural networks with external memory has a rich history. **Neural Turing Machines** [Graves et al., 2014] introduced differentiable read-write memory banks, allowing networks to learn algorithmic operations. The **Differentiable Neural Computer**

(DNC) [Graves et al., 2016] extended this with dynamic memory allocation and temporal link matrices that tracked write order, enabling more sophisticated memory management.

Memory Networks [Weston et al., 2015] and their end-to-end variants [Sukhbaatar et al., 2015] demonstrated that explicit memory modules could improve question-answering and reasoning tasks by attending over a set of stored memories. These architectures laid important groundwork but operate over fixed, pre-populated memory stores without temporal dynamics.

2.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) [Lewis et al., 2020] has emerged as the dominant paradigm for grounding LLM outputs in external knowledge. RAG systems retrieve relevant documents from a vector store at inference time and inject them into the prompt. Subsequent work has extended RAG with re-ranking [Nogueira & Cho, 2019], iterative retrieval [Trivedi et al., 2023], and self-reflective retrieval [Asai et al., 2024]. However, RAG retrieves but does not *forget*—all documents remain equally accessible regardless of age or relevance, creating noise accumulation over time.

2.3 LLM-Based Agent Memory

Generative Agents [Park et al., 2023] demonstrated that LLM-based agents could maintain a stream of observations, reflect on them to form higher-level abstractions, and retrieve relevant memories to guide behavior. MemGPT [Packer et al., 2023] introduced an operating-system-inspired approach to LLM memory management with explicit paging between tiers. **Reflexion** [Shinn et al., 2023] showed that self-reflective memory could improve task performance over multiple trials.

These systems represent significant advances but share common limitations: they do not model user-specific behavioral signals, they lack formal temporal decay dynamics calibrated to cognitive science, and they do not address the specific challenge of building longitudinal user models that compound in value over time.

2.4 Cognitive Models of Memory

CTMA draws directly from established models in cognitive psychology. **Ebbinghaus's forgetting curve** [Ebbinghaus, 1885] established that memory retention decays exponentially, modulated by repetition and encoding depth. The **ACT-R architecture** [Anderson & Schooler, 1991, Anderson et al., 2004] formalized base-level activation as a function of recency and frequency, with spreading activation along associative links. **Levels of processing theory** [Craik & Lockhart, 1972] demonstrated that deeper semantic processing produces more durable memories. **Loftus and Palmer** [Loftus & Palmer, 1974] showed that memory is reconstructive rather than reproductive—an insight central to CTMA's design philosophy that forgetting is a feature, not a bug.

3 Theoretical Foundations

3.1 The Forgetting Curve as Computational Strategy

A critical insight of CTMA is that forgetting is not a failure mode but a computational necessity. An AI system that remembers everything with equal fidelity does not exhibit superior memory—it exhibits the absence of an information management strategy. In high-noise, longitudinal data environments, the ability to gracefully degrade low-value information while preserving high-value signals is essential for maintaining signal-to-noise ratio over time.

3.2 Information-Theoretic Formulation

Let $\mathcal{U}$ represent the universe of all observations about a user across all interactions. Let $\mathcal{R}(t) \subseteq \mathcal{U}$ represent the set of memories retained at time t. Let $\mathcal{F}$ denote the distribution of future user information needs. The optimal memory management problem is:

Definition 1 (Optimal Memory Curation)**.** *Given capacity constraint $C(t)$ and retrieval budget B, find:*

$$\mathcal{R}^*(t) = \arg\max_{\mathcal{R}(t)} I(\mathcal{R}(t); \mathcal{F}) \quad \textit{subject to} \quad |\mathcal{R}(t)| \leq C(t), \quad \sum_{r \in \mathcal{R}(t)} \textit{cost}(r) \leq B \tag{1}$$

where $I(\cdot;\cdot)$ denotes mutual information.

This formulation reveals that optimal memory management requires active curation—precisely the function performed by CTMA's cascading decay mechanism. Since $\mathcal{F}$ is unknown, we approximate it using the empirically-validated prior from Anderson & Schooler [1991]: information that has been accessed recently and frequently is more likely to be needed again.

3.3 Three-Tier Cognitive Mapping

We identify three tiers of human memory that map directly onto our architecture:

(i) **Working memory** (ephemeral): High-bandwidth, extremely limited capacity, session-scoped. Corresponds to the active state of a single conversation.

(ii) **Consolidation zone** (cascading): Intermediate capacity, subject to reinforcement through revisitation, emotional tagging, and contextual relevance. Memories exist in dynamic flux.

(iii) **Long-term memory** (permanent): Massive capacity, deeply encoded facts, identity-level knowledge, core attributes. Resistant to decay.

Between the extremes of working and long-term memory lies the *cascading zone*—where memories compete for survival, strengthening through reinforcement and weakening through neglect.

4 The CTMA Framework

4.1 System Overview

CTMA organizes memory into three distinct tiers, each with its own retention policy, access characteristics, and promotion/demotion dynamics. Table 1 summarizes the tier properties.

Table 1: Comparison of the three CTMA memory tiers.

Property	**Ephemeral**	**Cascading**	**Permanent**
Retention	Session-scoped	Days to months	Indefinite
Decay function	Hard expiry	Modulated exponential	None (immune)
Capacity	∼50K tokens	Unbounded (indexed)	Bounded (∼10K items)
Write trigger	Every utterance	Extraction pipeline	Promotion from cascading
Access pattern	Sequential	Similarity + salience	Always injected
Examples	Current dialog turns	Preferences, projects	Name, identity, core goals

4.2 Tier 1: Ephemeral Memory

The ephemeral tier holds the full dialog history, injected context, tool outputs, and transient artifacts of a single session. At session termination, the ephemeral tier is flushed through an *extraction pipeline* that identifies candidate memories for promotion to the cascading tier. The extraction pipeline employs named entity recognition, semantic classification, and LLM-based summarization to identify candidates. Each candidate is tagged with metadata: timestamp, source session ID, semantic category $c \in \mathcal{C}$, and initial importance score S_0.

Definition 2 (Semantic Category Set)**.** $\mathcal{C} = \{$*PREFERENCE*, *FACT*, *DECISION*, *EMOTION*, *GOAL*, *RELATIONSHIP*, *II*

4.3 Tier 2: The Cascading Memory Zone

The cascading tier is the heart of the CTMA architecture. Memories here exist in dynamic flux—strengthening through reinforcement, weakening through neglect, and competing for survival.

4.3.1 The Temporal Decay Function

Each memory m in the cascading tier is assigned a *salience score* $S(m,t)$ that evolves over time:

$$\boxed{S(m,t) = S_0(m) \cdot e^{-\lambda(m)\cdot\Delta t} + \alpha \cdot F(m,t) + \beta \cdot B(m,t) + \gamma \cdot C(m,t)} \tag{2}$$

where:

- $S_0(m) \in [0,1]$ is the initial importance score assigned at extraction time, determined by semantic category, emotional valence, and contextual novelty.
- $\lambda(m) > 0$ is the memory-specific decay rate, inversely proportional to initial importance.
- $\Delta t = t - t_{\text{last}}(m)$ is the elapsed time since the memory's last access or reinforcement event.
- $F(m,t)$ is the *frequency reinforcement* term (Section 4.3.2).
- $B(m,t)$ is the *behavioral signal* term (Section 4.3.3).
- $C(m,t)$ is the *contextual co-activation* term (Section 4.3.4).
- $\alpha, \beta, \gamma \geq 0$ are weighting hyperparameters tuned per user cohort.

This formulation is directly inspired by the ACT-R base-level activation equation [Anderson et al., 2004], which posits that memory activation is a function of recency and frequency plus associative spreading activation.

4.3.2 Frequency Reinforcement

The frequency term models the spacing effect [Cepeda et al., 2006]—memories accessed across multiple spaced sessions consolidate more strongly than those accessed in rapid succession:

$$F(m,t) = \log\left(1 + \sum_{j=1}^{n(m)} (t - t_j)^{-d}\right) \tag{3}$$

where $n(m)$ is the number of prior accesses, t_j is the timestamp of the j-th access, and $d \approx 0.5$ is the temporal discount parameter from the ACT-R literature [Anderson & Schooler, 1991].

4.3.3 Behavioral Signal Term

The behavioral term aggregates discrete user signals using a time-weighted exponential moving average:

$$B(m,t) = \sum_i w_i \cdot s_i \cdot e^{-\mu(t-t_i)} \tag{4}$$

where w_i is the weight for signal type i (see Section 6), s_i is the signal magnitude, t_i is the signal timestamp, and μ is the signal decay parameter. This ensures that isolated signals gradually lose influence unless reinforced.

4.3.4 Contextual Co-Activation

The co-activation term implements spreading activation along the memory knowledge graph:

$$C(m,t) = \sum_{m' \in \mathcal{N}(m)} w_{m \to m'} \cdot \mathbb{1}[\text{accessed}(m', t, \tau)] \tag{5}$$

where $\mathcal{N}(m)$ is the set of memories connected to m by typed edges, $w_{m \to m'}$ is the edge weight, and $\mathbb{1}[\text{accessed}(m', t, \tau)]$ is an indicator function that equals 1 if memory m' was accessed within the time window $[t - \tau, t]$.

4.3.5 Decay Rate Modulation

The decay rate $\lambda(m)$ is dynamically adjusted based on the memory's behavioral history:

$$\lambda(m) = \frac{\lambda_{\text{base}}(c_m)}{1 + \delta \cdot \log(1 + n_{\text{access}}(m)) + \varepsilon \cdot I(m)} \tag{6}$$

where $\lambda_{\text{base}}(c_m)$ is the default decay rate for semantic category c_m, $n_{\text{access}}(m)$ is the total access count, $I(m)$ is the importance score, and δ, ε are tuning parameters.

Proposition 1 (Monotonic Decay Deceleration)**.** *Under the modulation scheme of Eq. 6, for any memory m with $n_{access}(m) \geq 1$:*

$$\lambda(m) < \lambda_{base}(c_m)$$

Moreover, $\lambda(m)$ is monotonically decreasing in both $n_{access}(m)$ and $I(m)$. Frequently accessed or high-importance memories decay strictly slower than baseline.

4.3.6 Promotion and Demotion

Definition 3 (Tier Transitions)**.** *Let $T_\uparrow$ and $T_\downarrow$ denote promotion and demotion thresholds respectively, with $T_\uparrow > T_\downarrow$.*

Promotion *(Cascading → Permanent): Memory m is promoted if:*

$$S(m,t') > T_\uparrow \quad \forall\, t' \in [t - \Delta_{sustain},\ t] \quad \wedge \quad |\{b \in \mathcal{B}(m) : b.type \neq b'.type\}| \geq 3 \tag{7}$$

where $\Delta_{sustain} = 30$ days (default) and $\mathcal{B}(m)$ is the set of distinct behavioral signal types received.

Demotion *(Cascading → Archive): Memory m is archived if $S(m,t) < T_\downarrow$.*

The thresholds $T_\uparrow$ and $T_\downarrow$ are calibrated per semantic category, since identity-related memories have inherently higher baseline salience than preference-level memories.

4.4 Tier 3: Permanent Memory

Permanent memories are immune to temporal decay and are always included in the retrieval set. However, they are not immutable: they can be overridden by explicit user corrections or invalidated by contradictory evidence from the cascading tier, in which case they are flagged for review. The permanent tier is capacity-bounded ($\sim$10,000 items per user). When approaching capacity, a consolidation process merges semantically overlapping memories and prunes redundant entries.

5 Data Model and Knowledge Graph

5.1 Memory Node Schema

Each memory is represented as a node in a directed, typed knowledge graph. Table 2 presents the schema.

Table 2: Memory node schema for the CTMA knowledge graph.

Field	Type	Description
`memory_id`	UUID	Unique identifier
`user_id`	UUID	Owning user
`tier`	Enum	EPHEMERAL | CASCADING | PERMANENT
`content`	Text	Natural language memory content
`embedding`	$\mathbb{R}^{768}$	Semantic embedding vector
`semantic_category`	$c \in \mathcal{C}$	Category from Definition 2
`salience_score`	$\mathbb{R}_{[0,1]}$	Current $S(m,t)$ value
`initial_importance`	$\mathbb{R}_{[0,1]}$	$S_0(m)$ at extraction
`decay_rate`	$\mathbb{R}_{>0}$	Current $\lambda(m)$
`access_count`	$\mathbb{Z}_{\geq 0}$	Total retrieval count
`last_accessed`	Timestamp	Most recent access
`created_at`	Timestamp	Extraction timestamp
`source_sessions`	UUID[]	Contributing session IDs
`behavioral_signals`	JSON[]	{type, timestamp, weight} tuples
`edges`	Edge[]	Typed relationships to other nodes

5.2 Edge Types and Associative Activation

Memory nodes are connected by typed, weighted edges:

- SUPPORTS: Memory A provides evidence for B.
- CONTRADICTS: A and B are in tension.
- SUPERSEDES: A replaces an outdated version of B.
- CO-OCCURS: A and B were extracted from the same session.
- CAUSES: A is a causal antecedent of B.
- GENERALIZES: A is a higher-level abstraction of B.

When a memory is accessed, its activation spreads along these edges with a decay factor proportional to edge weight (Eq. 5), naturally surfacing related memories without exhaustive search.

6 Behavioral Signal Integration

6.1 Signal Taxonomy

The $B(m, t)$ term is computed from a weighted sum of discrete behavioral signals:

Table 3: Behavioral signal types and their weights.

Signal Type	**Weight** w_i	**Magnitude**	**Description**
Explicit correction	2.0	Binary	User corrects system output
Revisitation	1.5	$\log(n_{\text{sessions}})$	Referenced across independent sessions
Engagement duration	1.0	Normalized time	Extended interaction with related content
Emotional valence	1.2	Sentiment score	Strong emotional response detected
Explicit save/pin	2.5	Binary	User explicitly saves information
Contradiction	1.8	Binary	New info contradicts existing memory

6.2 Anti-Gaming Measures

The exponential moving average formulation (Eq. 4) provides inherent resistance to signal manipulation. Additionally, we apply:

(i) **Signal rate limiting**: No more than k signals of the same type are counted per session.

(ii) **Diversity bonus**: The promotion criterion (Eq. 7) requires signals from ≥ 3 distinct types, preventing promotion through a single repeated action.

(iii) **Temporal spacing**: Signals within a short burst window (< 60s) are deduplicated.

7 Context Assembly and Retrieval

7.1 Multi-Stage Retrieval Pipeline

At inference time, CTMA assembles a context payload using a multi-stage pipeline:

Input: User query q, context budget K tokens
Output: Assembled context $\mathcal{C}_q$
$\mathcal{C}_q \leftarrow \emptyset$;
$K_{\text{remaining}} \leftarrow K$;

`// Stage 1: Permanent tier (unconditional)`
$\mathcal{P} \leftarrow \text{FormatProfile}(\text{PermanentMemories}(u))$;
$\mathcal{C}_q \leftarrow \mathcal{C}_q \cup \mathcal{P}$;
$K_{\text{remaining}} \leftarrow K_{\text{remaining}} - |\mathcal{P}|$;

`// Stage 2: Query embedding`
$\mathbf{q} \leftarrow \text{Encode}(q)$;

`// Stage 3: Cascading tier retrieval`
$\mathcal{M}_{\text{candidates}} \leftarrow \text{TopK}(\cos(\mathbf{q}, \mathbf{e}_m) \cdot S(m,t),\ k = K_{\text{remaining}}/\bar{L})$;

`// Stage 4: Associative expansion`
for $m \in \mathcal{M}_{candidates}$ **do**
  $\mathcal{M}_{\text{expanded}} \leftarrow \mathcal{M}_{\text{expanded}} \cup \{m' \in \mathcal{N}(m) : w_{m \to m'} > \theta\}$;
end

`// Stage 5: Deduplication and compression`
$\mathcal{M}_{\text{final}} \leftarrow \text{Deduplicate}(\text{Compress}(\mathcal{M}_{\text{candidates}} \cup \mathcal{M}_{\text{expanded}}))$;
$\mathcal{C}_q \leftarrow \mathcal{C}_q \cup \mathcal{M}_{\text{final}}$;

`// Stage 6: Tier tagging`
for $m \in \mathcal{C}_q$ **do**
  $m.\text{metadata} \leftarrow \{\text{tier}, S(m,t), t_{\text{last}}\}$;
end
return $\mathcal{C}_q$

Algorithm 1: CTMA Context Assembly Pipeline

7.2 Dynamic Context Budgeting

CTMA does not allocate a fixed proportion of the context window to memory. Instead, it uses a dynamic budgeting algorithm:

$$K_{\text{memory}} = K_{\text{total}} \cdot \sigma\left(\eta \cdot H(q) + \kappa \cdot \rho(\mathcal{R}_q) - \tau\right) \tag{8}$$

where $H(q)$ estimates query complexity via perplexity, $\rho(\mathcal{R}_q)$ is the density of relevant memories, σ is the sigmoid function, and η, κ, τ are learned parameters. Simple factual queries receive a thin memory slice; open-ended conversations trigger deeper retrieval.

8 The Longitudinal Dataset as Technical Moat

8.1 Compounding Data Advantage

The CTMA architecture generates a continuously compounding longitudinal dataset that grows richer with every user interaction. Each interaction produces not just a conversation record, but a delta to the user's knowledge graph—new nodes, updated salience scores, new edges, and refined decay parameters. This creates a flywheel:

$$\text{Value}(t) = f\left(\int_0^t \text{Interactions}(\tau) \cdot \text{MemoryQuality}(\tau)\, d\tau\right) \tag{9}$$

where value grows superlinearly with accumulated interaction history. A competitor entering the market starts with $\text{Value}(0) = 0$, and the integral cannot be shortcut.

8.2 Four Axes of Defensibility

We identify four axes of competitive advantage:

(1) **Temporal depth.** Each day adds irreplaceable temporal context. The system learns not just what a user prefers, but *how their preferences evolve*.

(2) **Behavioral signal density.** The richness of corrections, revisitations, and emotional markers cannot be bootstrapped from static data or synthetic generation.

(3) **Network effects.** As the system processes more users, category-level decay models improve, benefiting all users.

(4) **Switching costs.** A user's longitudinal memory graph is not portable to competing systems.

8.3 Privacy-Preserving Analysis

User memory graphs are encrypted at rest, sharded by user, and never used for cross-user training without explicit consent. Differential privacy guarantees ($\varepsilon = 1.0$ per quarterly cycle) are applied to all aggregate analyses.

9 Implementation

9.1 System Components

CTMA is deployed as a microservices architecture comprising: (i) an *extraction service* processing session transcripts through NLP pipelines; (ii) a *graph store* built on a temporal property graph database with full versioning; (iii) a *decay engine* running asynchronous salience updates (hourly batch, real-time on interaction); (iv) a *retrieval service* executing the multi-stage pipeline of Algorithm 1; and (v) a *behavioral signal collector* instrumenting the UI to capture interaction events.

9.2 Scalability

User graphs are independent and sharded by user ID, enabling embarrassingly parallel processing. The vector index uses a Hierarchical Navigable Small World (HNSW) graph [Malkov & Yashunin, 2018] partitioned per user. At current scale, the system processes approximately 50M memory updates per day with p99 retrieval latency under 40ms.

10 Empirical Observations from Prior Iterations

The CTMA architecture has undergone multiple iterations since its initial conception. Early versions of the core memory system—including temporal decay, behavioral signal integration, and tiered storage—were deployed within predecessor platforms (including Kaya Tech) and earlier builds of `gutbut.com` between 2023 and 2024. During this period, approximately 2,500 active users interacted with the system over a 90-day observation window. The platform has since been rebuilt from the ground up to incorporate the formal architecture described in this paper, and a rigorous validation study on the current implementation is underway.

We report the following *directional findings* from the prior deployment. These observations informed the design of the current CTMA architecture and are presented here as indicative evidence rather than formally controlled experimental results. A comprehensive empirical evaluation with ablation studies, statistical controls, and reproducible methodology is planned for a follow-up publication.

Table 4: Directional observations from prior CTMA iterations (2023–2024, $n \approx 2{,}500$). These results are from an earlier platform build and are presented as indicative findings, not formally controlled experiments.

Metric	**Description**	**Observed Result**
Extraction precision	Memory-worthy info correctly identified	∼87%
Extraction recall	Memory-worthy info captured vs. manual review	∼73%
Decay–relevance correlation	Spearman ρ between $S(m,t)$ and user-rated relevance	∼0.71
Satisfaction vs. no memory	Internal A/B test, CTMA vs. memoryless baseline	∼+34%
Satisfaction vs. naïve RAG	Internal A/B test, CTMA vs. all-memories-no-decay	∼+19%
30-day retention (high graph)	Users with > 200 memory nodes vs. < 50 nodes	∼2.4×
Promotion accuracy	Permanent-tier memories validated at 60 days	∼91%

While these findings are directional, two observations are particularly noteworthy. First, the consistent gap between CTMA and naïve RAG (∼19% satisfaction improvement) suggests that intelligent forgetting—not merely retrieval—is the critical differentiator. Second, the strong directional correlation between algorithmic salience and user-perceived relevance ($\rho \approx 0.71$) provides initial evidence that the decay function captures genuine user priorities rather than arbitrary temporal patterns. Formal validation of these findings under controlled conditions is a primary objective of ongoing work.

11 Future Directions

Several extensions are under active development:

(1) **Multi-modal memory.** Extending the knowledge graph to encode images, audio patterns, and structured data alongside text.

(2) **Federated memory.** Enabling users to selectively share subgraphs with other users or agents while preserving privacy.

(3) **Predictive pre-loading.** Using temporal patterns to anticipate which memories will be needed and pre-compute context payloads.

(4) **Meta-learned decay.** Learning user-specific λ parameters from interaction history rather than cohort-level hyperparameters, enabling personalized forgetting curves.

(5) **Formal convergence analysis.** Proving conditions under which the salience function converges to a stable distribution over the memory graph.

12 Conclusion

The Cascading Temporal Memory Architecture represents a fundamental advance in how AI systems maintain persistent, adaptive, and personalized memory. By drawing on cognitive psychology, neuroscience, and information theory, CTMA moves beyond context windows, naïve RAG, and static memory stores to create a living memory system that mirrors the dynamics of human cognition—remembering what matters, forgetting what doesn't, and continuously adapting to each user's evolving needs.

The implementation within the `gutbut.com` platform demonstrates that this architecture is practically deployable at scale, and that the longitudinal dataset it generates constitutes a powerful and defensible technical moat. As AI systems are increasingly expected to serve as long-term partners and collaborators, the ability to truly remember—and to forget gracefully—will become a defining differentiator.

Correspondence: `research@gutbut.com` | https://gutbut.com

References

Anderson, J. R. & Schooler, L. J. (1991). Reflections of the environment in memory. *Psychological Science*, 2(6):396–408.

Anderson, J. R., Bothell, D., Byrne, M. D., Douglass, S., Lebiere, C., & Qin, Y. (2004). An integrated theory of the mind. *Psychological Review*, 111(4):1036–1060.

Anthropic. (2024). The Claude model family. Technical report, Anthropic.

Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *ICLR*.

Cepeda, N. J., Pashler, H., Vul, E., Wixted, J. T., & Rohrer, D. (2006). Distributed practice in verbal recall tasks: A review and quantitative synthesis. *Psychological Bulletin*, 132(3):354–380.

Craik, F. I. M. & Lockhart, R. S. (1972). Levels of processing: A framework for memory research. *Journal of Verbal Learning and Verbal Behavior*, 11(6):671–684.

Ebbinghaus, H. (1885). *Memory: A Contribution to Experimental Psychology*. Teachers College, Columbia University (1913 translation).

Graves, A., Wayne, G., & Danihelka, I. (2014). Neural Turing machines. *arXiv preprint arXiv:1410.5401*.

Graves, A., Wayne, G., Reynolds, M., Harley, T., Danihelka, I., Grabska-Barwińska, A., ... & Hassabis, D. (2016). Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):471–476.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*.

Loftus, E. F. & Palmer, J. C. (1974). Reconstruction of automobile destruction: An example of the interaction between language and memory. *Journal of Verbal Learning and Verbal Behavior*, 13(5):585–589.

Malkov, Y. A. & Yashunin, D. A. (2018). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836.

Nogueira, R. & Cho, K. (2019). Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*.

OpenAI. (2023). GPT-4 technical report. *arXiv preprint arXiv:2303.08774*.

Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*.

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. In *UIST*.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*.

Sukhbaatar, S., Szlam, A., Weston, J., & Fergus, R. (2015). End-to-end memory networks. In *NeurIPS*.

Team, G., Anil, R., Borgeaud, S., Wu, Y., Alayrac, J.-B., Yu, J., ... & others. (2023). Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.

Trivedi, H., Balasubramanian, N., Khot, T., & Sabharwal, A. (2023). Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *ACL*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *NeurIPS*.

Weston, J., Chopra, S., & Bordes, A. (2015). Memory networks. In *ICLR*.